

Exercise series 6: Active Databases



ADReM
Advanced Database Research & Modelling
University of Antwerp

 Universiteit
Antwerpen

Problem 1

Create in PostgreSQL a schema based on the following description.

- A student has a first and last name. The combination of first and last name must be unique, and both fields are mandatory. A student can also be identified by a unique enrolment number which must be between 10*000 and 99*999.
- A course has a unique name and id, which must be a positive integer. A course also has a number of student points. The number of student points is either 3, 6, 9 or 12.
- An academic year consist of a starting year and ending year. The starting year should be a year before the ending one, and both should be between 2000 and 2100.
- A professor is identified by the combination of first and last name.
- A course offering is composed of a course, an academic year and a professor. We also store the semester: either 'first' or 'second'. The combination of year, semester and course identifies the course offering.
- If student enrolment number, professor name or course id is updated, all references are updated.
- If a student is deleted, delete all enrolments of that student.
- If a professor is deleted set reference to null.
- If a course is deleted, delete all enrolments and teaching assignments.

Problem 1

```
CREATE TABLE Student (  
    first_name TEXT NOT NULL,  
    last_name TEXT NOT NULL,  
    PRIMARY KEY (first_name, last_name),  
    stud_no INT UNIQUE NOT NULL,  
    CHECK (stud_no >= 10000 AND stud_no < 100000)  
);
```

```
CREATE DOMAIN pts INT CHECK( VALUE IN (3,6,9,12));
```

```
CREATE TABLE Course (  
    cid INT PRIMARY KEY CHECK (cid > 0),  
    name TEXT UNIQUE NOT NULL ,  
    stud_points pts NOT NULL  
);
```

- A student has a first and last name. The combination of first and last name must be unique, and both fields are mandatory. A student can also be identified by a unique enrolment number which must be between 10*000 and 99*999.
- A course has a unique name and id, which must be a positive integer. A course also has a number of student points. The number of student points is either 3, 6, 9 or 12.



Problem 1

```
CREATE TABLE Academic_year (  
  start_year INT NOT NULL,  
  end_year INT PRIMARY KEY,  
  CONSTRAINT right_time CHECK  
  (  
    end_year - start_year = 1 AND  
    start_year >= 2000 AND  
    end_year <= 2100  
  )  
);
```

```
CREATE TABLE Professor (  
  first_name TEXT,  
  last_name TEXT,  
  PRIMARY KEY(first_name, last_name)  
);
```

- An academic year consist of a starting year and ending year. The starting year should be a year before the ending one, and both should be between 2000 and 2100.
- A professor is identified by the combination of first and last name.



Problem 1

```
CREATE TABLE Course_offering (  
  oid SERIAL PRIMARY KEY,  
  cid INT REFERENCES Course(cid) ON UPDATE CASCADE ON DELETE CASCADE,  
  year INT REFERENCES Academic_year,  
  prof_first_name TEXT,  
  prof_last_name TEXT,  
  FOREIGN KEY (prof_first_name, prof_last_name) REFERENCES Professor ON UPDATE CASCADE ON DELETE SET NULL,  
  semester TEXT CHECK (semester IN ('first', 'second')),  
  UNIQUE (cid, year, semester)  
);
```

- A course offering is composed of a course, an academic year and a professor. We also store the semester: either 'first' or 'second'. The combination of year, semester and course identifies the course offering.



Problem 1

```
CREATE TABLE Course_offering (  
  oid SERIAL PRIMARY KEY,  
  cid INT REFERENCES Course(cid) ON UPDATE CASCADE ON DELETE CASCADE,  
  year INT REFERENCES Academic_year,  
  prof_first_name TEXT,  
  prof_last_name TEXT,  
  FOREIGN KEY (prof_first_name, prof_last_name) REFERENCES Professor ON UPDATE CASCADE ON DELETE SET NULL,  
  semester TEXT CHECK (semester IN ('first', 'second')),  
  UNIQUE (cid, year, semester)  
);
```

- A course offering is composed of a course, an academic year and a professor. We also store the semester: either 'first' or 'second'. The combination of year, semester and course identifies the course offering.
- If student enrolment number, professor name or course id is updated, all references are updated.



Problem 1

```
CREATE TABLE Course_offering (  
  oid SERIAL PRIMARY KEY,  
  cid INT REFERENCES Course(cid) ON UPDATE CASCADE ON DELETE CASCADE,  
  year INT REFERENCES Academic_year,  
  prof_first_name TEXT,  
  prof_last_name TEXT,  
  FOREIGN KEY (prof_first_name, prof_last_name) REFERENCES Professor ON UPDATE CASCADE ON DELETE SET NULL,  
  semester TEXT CHECK (semester IN ('first', 'second')),  
  UNIQUE (cid, year, semester)  
);
```

- A course offering is composed of a course, an academic year and a professor. We also store the semester: either 'first' or 'second'. The combination of year, semester and course identifies the course offering.
- If student enrolment number, professor name or course id is updated, all references are updated.
- If a professor is deleted set reference to null.
- If a course is deleted, delete all enrolments and teaching assignments.



Problem 1

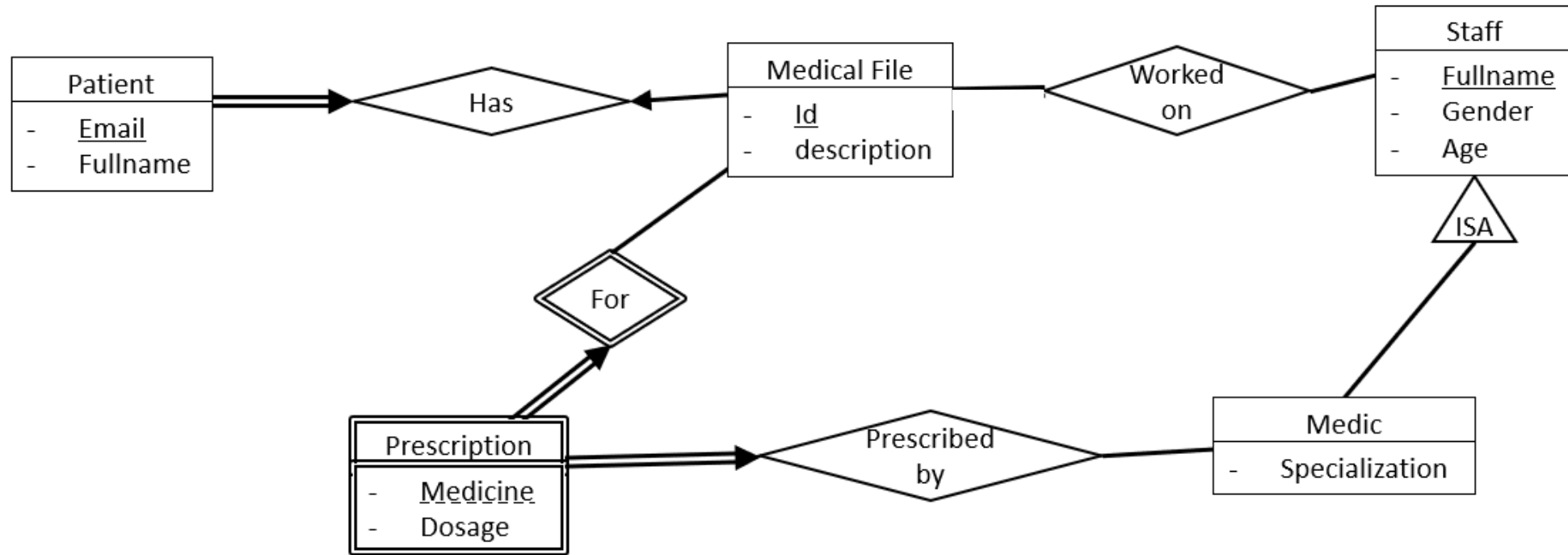
- If a student is deleted, delete all enrolments of that student.
- If a course is deleted, delete all enrolments and teaching assignments.

```
CREATE TABLE Enrollment (  
  oid INT REFERENCES Course_offering ON UPDATE CASCADE ON DELETE CASCADE,  
  stud_no INT REFERENCES Student(stud_no) ON UPDATE CASCADE ON DELETE CASCADE,  
  PRIMARY KEY (oid, stud_no)  
);
```



Problem 2

Consider the following ER diagram for a private clinic:



Problem 2

1. Translate the ER diagram into a relational model

2. Use checks, constraints and domains to enforce the followings constraints:

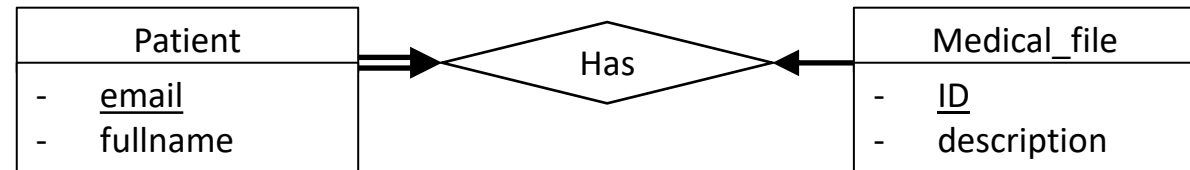
- Dosage is in *mg* and should be an integer greater than zero.
- A patient can be identified by his/her name
- Staff age must be above or equal 18 y.o.
- If a medic is removed then his/her prescriptions are not valid anymore and must be removed. If a medic changes name then it is updated.
- If a staff member is removed or changes name then all the entries with a references to that member are deleted/updated also
- If a patient removes his/her account then his/her medical file is kept but without the reference.



Problem 2

```
CREATE TABLE Medical_file (  
    mid INT PRIMARY KEY,  
    description TEXT  
);
```

```
CREATE TABLE Patient (  
    email TEXT PRIMARY KEY,  
    fullname TEXT UNIQUE NOT NULL,  
    mid INT NOT NULL UNIQUE REFERENCES Medical_file  
);
```



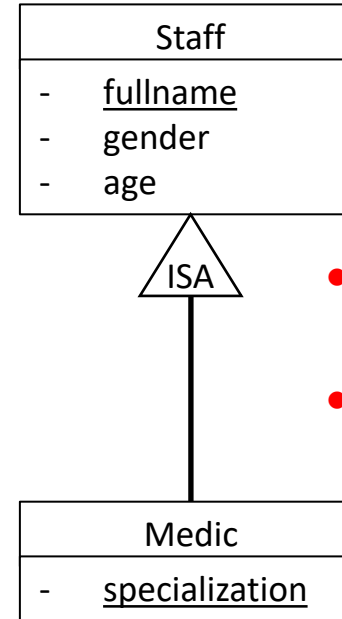
- A patient can be identified by his/her name
- If a patient removes his/her account then his/her medical file is kept but without the reference.



Problem 2

```
CREATE TABLE Staff (  
    fullname TEXT PRIMARY KEY,  
    gender TEXT,  
    age INT CHECK (age >= 18)  
);
```

```
CREATE TABLE Medic (  
    fullname TEXT PRIMARY KEY,  
    FOREIGN KEY (fullname) REFERENCES Staff ON DELETE CASCADE ON UPDATE CASCADE,  
    specialization TEXT  
);
```

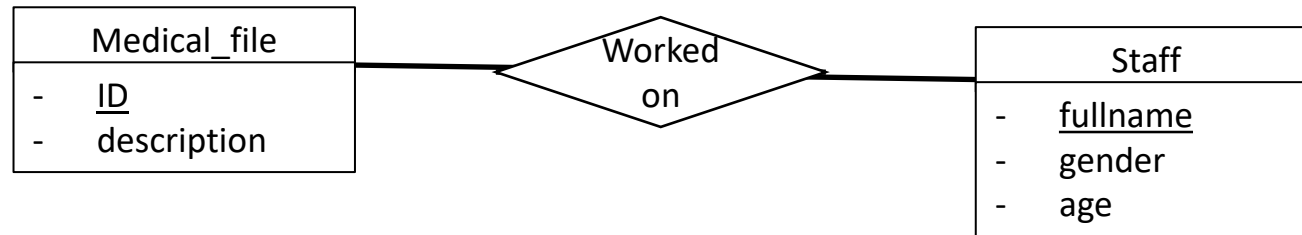


- Staff age must be above or equal 18 y.o.
- If a staff member is removed or changes name then all the entries with a references to that member are deleted/updated also



Problem 2

```
CREATE TABLE Worked_on (  
    mid INT REFERENCES Medical_file,  
    fullname TEXT REFERENCES Staff ON DELETE CASCADE ON UPDATE CASCADE,  
    PRIMARY KEY (mid, fullname)  
);
```



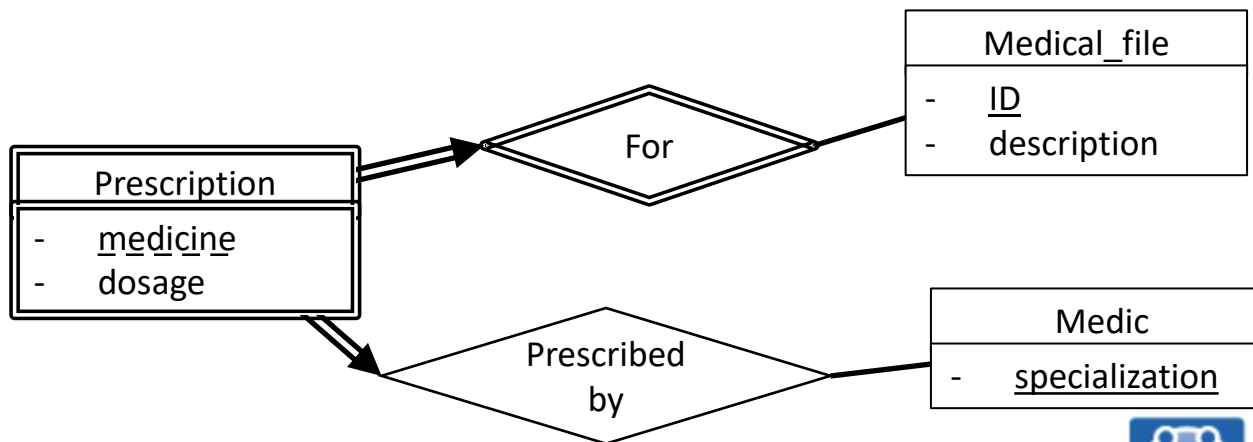
- If a staff member is removed or changes name then all the entries with a references to that member are deleted/updated also



Problem 2

```
CREATE DOMAIN mg INT CHECK (value > 0);
```

```
CREATE TABLE Prescription (  
    medicine TEXT,  
    mid INT REFERENCES Medical_file,  
    PRIMARY KEY (medicine, mid),  
    fullname TEXT NOT NULL REFERENCES Medic ON DELETE CASCADE ON UPDATE CASCADE,  
    dosage mg NOT NULL  
);
```



- Dosage is in *mg* and should be an integer greater than zero.
- If a medic is removed then his/her prescriptions are not valid anymore and must be removed. If a medic changes name then it is updated.

